Software Design Document (SDD)

Team Name:

ProjectMasters

Project Title:

National Freelance and Skill Verification Platform

Module:

Communication Module

Team Members:

Tanees Fatima 23i-3003

Nisha Zafran 23i-3023

Haifa Yousaf 23i-3072

Amna Shahzad 23i-3073

Section:

BSE-6B

Date:

28/03/26

Table of Contents

1 Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Overview 3
1.4 Reference Material 4
1.5 Definitions and Acronyms 4
2 System Overview 4
3 System Architecture 5
3.1 Architectural Design 5
3.2 Security & Encryption Design 5
3.3 Decomposition Description (OO Design) 6
3.4 Design Rationale 6
4 Data Design 7
4.1 Data Description 7
4.2 Data Dictionary 7
5 Component Design 8
6 Human Interface Design 11
6.1 Overview of User Interface 11
6.2 Screen Images 11
6.3 Screen Objects and Actions 12
7 Requirements Matrix 12
8 Appendices 12

1 Introduction

1.1 Purpose

This document describes the architecture and detailed design of the Communication Module, enabling developers to implement messaging, file sharing, notifications, meeting integration, and group communication.

1.2 Scope

The Communication Module enables secure interaction between freelancers and clients.

- Real-time messaging
- Multimedia management (file, images, videos sharing)
- Generate links for online meetings (zoom or google meet)
- One to one communication and Group chats
- Message status indicators (sent/read)
- Message Storage
- Basic Communication Interface Support
- Notification of received messages
- API/Service for Messaging
- End to end encryption of messages

1.3 Overview

This document includes:

- Architecture design
- Data structures
- Component-level design
- UI design
- Requirement mapping

1.4 Reference Material

- Project Business Case (Assignment 1)
- IEEE Std 1016-2009
- Refined Module Document (by integration group)
- SRS Document
- Project Management Plan
- Requirement Management Plan

1.5 Definitions and Acronyms

- UI: User Interface
- API: Application Programming Interface
- DB: Database
- RTM: Requirements Traceability Matrix
- WebSocket: Protocol for real-time communication

2 System Overview

The Communication Module allows users to:

- Send/receive messages in real-time (1-1 and group)
- Share files, image and videos
- Create meeting links
- Access chat history
- Perform group communication
- Receive notification

It interacts with:

- Module 1 (User Identity, Profile & Portfolio Management)

3 System Architecture

3.1 Architectural Design

The module follows a client-server architecture with real-time communication support (WebSocket model).

Main Components:

- Frontend UI (Chat Interface)
- Backend API Server
- WebSocket Server (Real-time messaging)
- Database (Messages)
- Media (file, image and video) Storage System
- Notification services

Interaction Flow:

User -> UI -> WebSocket/API -> Server -> Database -> Receiver

- User sends message to Frontend
- Frontend to WebSocket Server
- Server processes and then stores in DB
- Receiver gets message instantly

3.2 Security & Encryption Design

To ensure secure communication between users, the Communication Module implements SSL/TLS encryption.

Key Points:

- All data transmitted between client and server is encrypted using SSL/TLS.
- Ensures confidentiality of messages (prevents unauthorized access).
- Protects against man-in-the-middle attacks.

Secures:

- Text messages
- Media files (images, videos, documents)

- Meeting links
- User identifiers (Sender ID, Receiver ID)

Implementation Details:

- HTTPS protocol will be used for all API communication.
- Secure WebSocket (WSS) will be used instead of plain WebSocket.
- SSL/TLS certificates will be configured on the server.
- Encryption works in transit (data is protected while being sent over the network).

3.3 Decomposition Description (OO Design)

Main Classes (can be more):

- User
- Message
- ChatRoom
- Media (files, image and videos)
- Meeting (Link)
- Notification

Sequence Flow (Message Sending):

User ->UI ->WebSocket ->Server ->DB ->Receiver

Sequence (Meeting Link):

User ->UI ->Server ->Generate Link ->Send Message ->Receiver

Sequence (Group Chat):

User ->Create ChatRoom ->Add Members ->Send Message ->All receive

3.4 Design Rationale

Chosen Architecture:

WebSocket-based real-time system

- Instant communication
- Efficient for chat apps
- Low latency

Alternatives Considered:

HTTP polling was rejected (slow, inefficient, high server load)

4 Data Design

4.1 Data Description

Entities:

- Users
- Messages
- ChatRooms
- MeetingLinks
- media
- Notifications

Storage:

- Links stored as message
- Messages stored in database
- Media (Files, image, videos) stored in cloud storage
- Notification stored for history

4.2 Data Dictionary

Entity	Attributes	Description
User	user_id, name, role	User
Message	msg_id, sender_id, content, timestamp, status	Chat
ChatRoom	room_id, participants, roles	Chat group
media	media_id, type, size	Shared file
Notification	notif_id, user_id, type	Alerts

MeetingLink	link_id, url, platform, timestamp	Meeting

5 Component Design

1. Send Message Function

```
function sendMessage(sender, receiver, message):
    validate sender
    attach timestamp
    set status = "sent"
    store in DB
    send via WebSocket
    update status = "delivered"
  receiver receives message
  notify receiver
```

2. Receive Message Function

```
function receiveMessage():
    listen WebSocket
    display message
    update status = "read"
```

3. Retry Message

```
function retryMessage(message):
    if delivery fails:
        retry sending
        log failure
```

4. Media Sharing

```
function uploadMedia(media):
    validatemedia(media)
    store file
    send as message
```

5. Get Chat History

```
function getChatHistory(chatroom):
    fetch messages
    sort by timestamp
    display
```

6. Media Validation

```
function validateMedia(file):
    check type (pdf, image, doc, video)
    check size limit
```

7. Create Meeting Link

```
function createMeetingLink(platform):
    generate link
    store link
    send in chat
```

8. Notification

```
function notifyUser(user):
    create notification
    send in-app alert
```

9. Group Chat

```
function createGroup(users):
    create chatroom
    assign roles
```

10. Send Group Message

```
function sendGroupMessage(chatroom, message):
    store message
    send to all participants
```

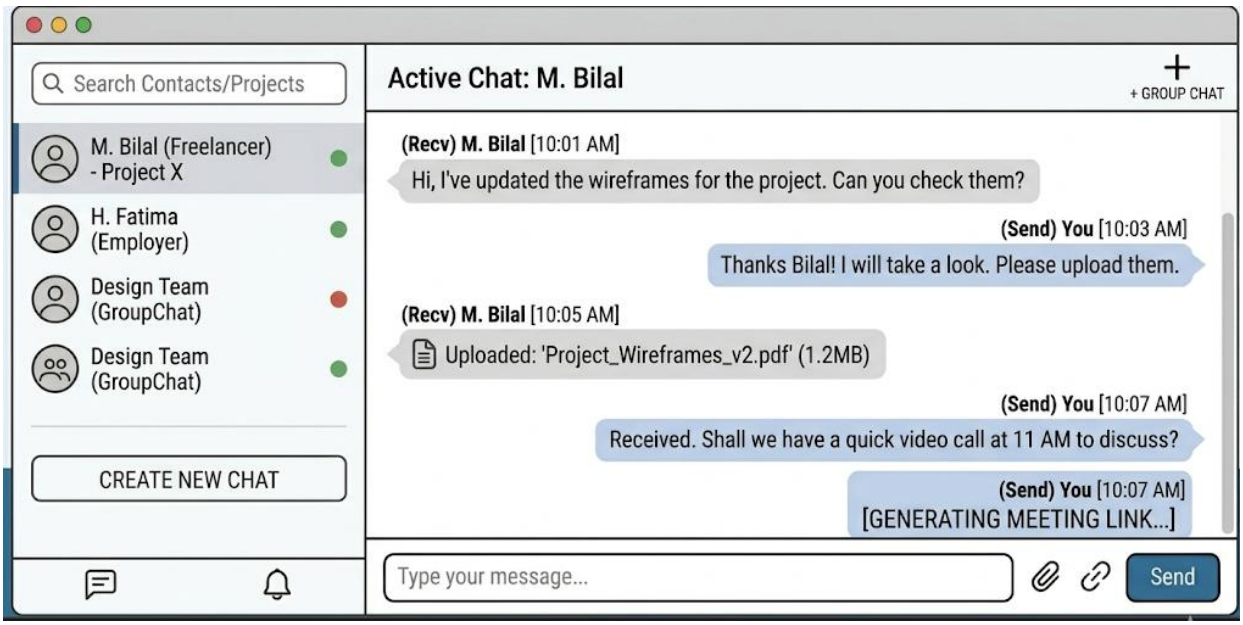
6 Human Interface Design

6.1 Overview of User Interface

Users can:

- Chat in real-time
- Share files, image and videos
- Create meetings (employers only)
- View history
- Use group chat

6.2 Screen Images



6.3 Screen Objects and Actions

Object	Action
Text Box	Type message
Send Button	Send message
clip Icon	Upload video, image, file
Chat Area	View History
Notification Icon	View alerts
Group Button	Create Group
Meet Button	Create link

7 Requirements Matrix

REQ ID	Requirement	Component
COM-FR-1	Send message	sendMessage
COM-FR-2	Real-time delivery	WebSocket
COM-FR-3	Timestamp	Message
COM-FR-4	Delivery status	Message status
COM-FR-5	Retry	retryMessage
COM-FR-6	media attach	uploadMedia
COM-FR-7	media types	validateMedia
COM-FR-8	Size limit	validateMedia
COM-FR-9	Meeting link	createMeetingLink
COM-FR-10	Zoom/Meet	API integration
COM-FR-11	Meeting history	DB
COM-FR-12	Send media history	Cloud DB
COM-FR-13	Notifications	notifyUser
COM-FR-14	Group creation	createGroup
COM-FR-15	Multi users	ChatRoom
COM-FR-16	Roles	ChatRoom roles

8 Appendices

Future Enhancements:

- Voice/Video Calling
- Notification System preference
- AI-based Chat or Bots
- Voice message